SharkFest'26 Europe

WebRTC Network Analysis Masterclass

Robert Hess, Alexander Müller



Sponsored by: PROFITAP DEEP NETWORK OBSERVABILITY · cPacket · TCP/IP training · endace Record. Respond.

CHAPTER 1

Introduction

Who we are and what we do today



Speaker Introduction

Alexander Müller - alexander.mueller@goto.com
Robert Hess - robert.hess@posteo.de

Building WebRTC based conferencing solutions running millions of calls (GoToMeeting/GoToWebinar/GoToTraining)

Alex - Developer / Architect

Robert - Network Troubleshooting / Wireshark / Focus Communication Protocols

ZIP file with

- Handouts
- Appendix with glossary, capture setup and product cheat sheets
- Sample Captures, Wireshark Profile, LUA plugins

⇒ **https://bit.ly/4mOt7oR**

> **Disclaimer:** All data captured for issue analysis will contain private information (PII) - think about it when asking for packet captures and when feeding them into arbitrary LLMs.

Agenda

Chapters

1. Introduction
2. WebRTC call setup in a nutshell
3. Analyzing a sample capture
4. Media Performance
5. Hands-on Live Capture
6. Dealing with problems
7. How products are using WebRTC

> **Do ask questions - like, lots of ...**

Timeline

- 9:00-10:40
- Break 20 mins
- 11:00-12:30
- Lunch 1h
- 13:30 -14:55
- Break 20 mins
- 15:15 - 16:45

Chapters in detail

1 - Introduction

- What does WebRTC cover and what is it used for
- Which communication modes
- What type of products use it
- **we look mainly at client-server setups**

2 - WebRTC call setup in a nutshell

- What happens before WebRTC kicks in - SDP
- Connectivity detection ICE/STUN/TURN

3 - Analyzing a sample capture

- Extra tooling - WebRTC Internals/ICEGrid/dump
- Walk through capture - setup and media flows
- Problem indicators

4 - Media Performance

- Cannot decrypt captures
- UDP vs TCP
- Packet loss & delay in a worldwide network
- I/O graph to visualize dips in performance
- Log files

Chapters in detail (cont.)

5 - Hands-on Live Capture

- Retup & run capture
- Analyze signaling (decrypted TCP) and media (UDP)
- Check a few selected captures

6 - Dealing with problems

- Packet filters
- Proxies, Zscaler, VPN, DNS
- DPI, TURN servers
- Checklists
- Security aspects

7 - How products are using WebRTC

The difference in how the various products use WebRTC

- MS Teams
- Zoom
- Cisco Webex
- (Google Meet)
- (GoTo)

Who are you?

Introduce yourself with 2 sentences

Do you have prior experience with WebRTC?

What’s your exposure with Wireshark?

What specifically do you want to learn today - about which product?

WebRTC basics



What is WebRTC?

- Set of tools designed to facilitate real-time audio, video and data communication over Internet
- Focus on browser as endpoint
- p2p and multi point communication
- JS API
- Covers connection setup, media handling, network control
- Does *not* cover initial signaling but does prescribe use of SDP
- Development started around 2010

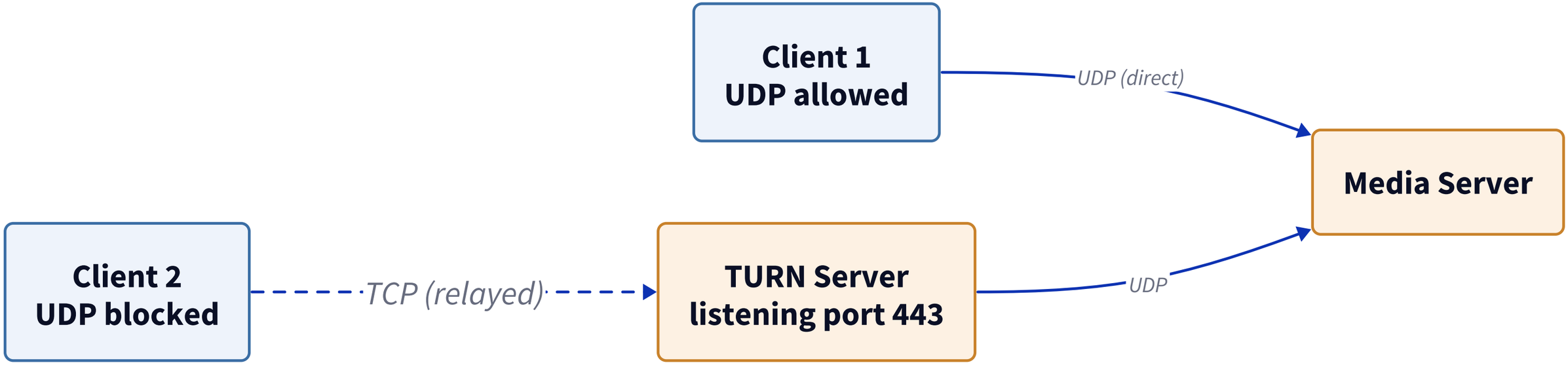
Where do we find WebRTC

> Traditionally used for a lot of the web conferencing platforms - these days new use case for **IoT** and **Voice AI** with lot of traction

- Predominantly in the browser (libwebrtc - maintained by Google)
- Some apps include WebRTC in various forms (Electron, react-native)
- Server-side implementations - Multipoint Control Units (MCU), Selective Forwarding Units (SFU), Gateways to other protocols
- Various SaaS services offer and use WebRTC
- Voice transmission for AI conversation input/output
- IoT devices like door bells with video or cameras

Basic components

- Client application in browser or standalone
- Communication servers (MCU, SFU, Gateways)
- STUN and TURN servers



WebRTC 1.0 Specification

WebRTC: Real-Time Communication in browsers

W3C recommendation since 2021 - https://www.w3.org/TR/webrtc/

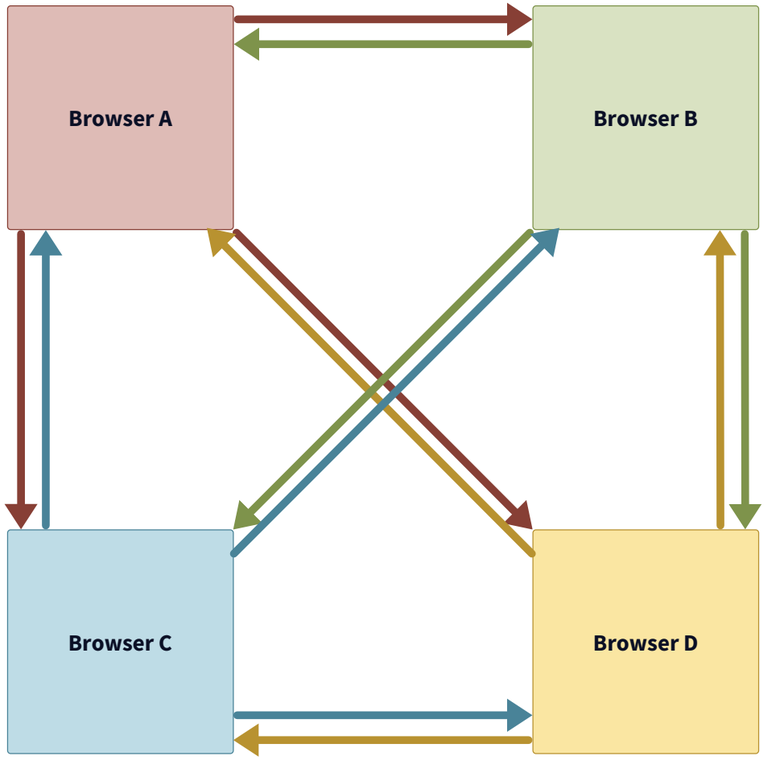
- Peer connections
  - Connection detection & negotiation via ICE (**I**nteractive **C**onnectivity **E**stablishment - RFC 8445)
  - Connection setup for media (SRTP) and data channels (SCTP)
  - Negotiation of connection parameters (codecs, bit rates)
- RTP Media API
  - Jitter buffer, flow control, error handling, audio signal processing, codecs
- Peer-to-Peer Data API

NOT defined are:

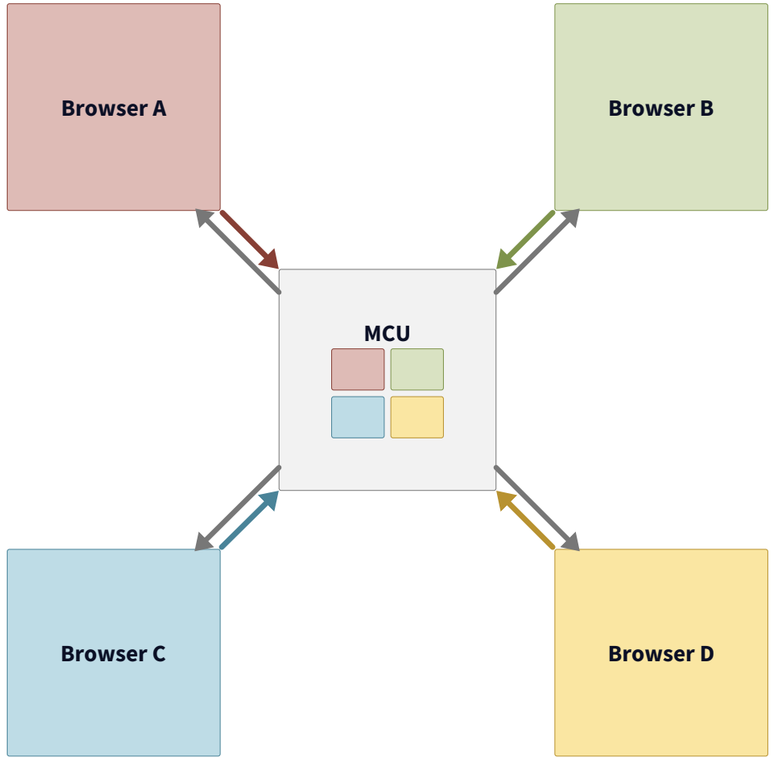
Initial call setup - how do I reach the other party in the call initially - this has to be done outside of WebRTC - but WebRTC expects the parameters as SDP structure.

Usually done by WebSocket and JSON or similar - like gRPC/Protobuf, XMPP

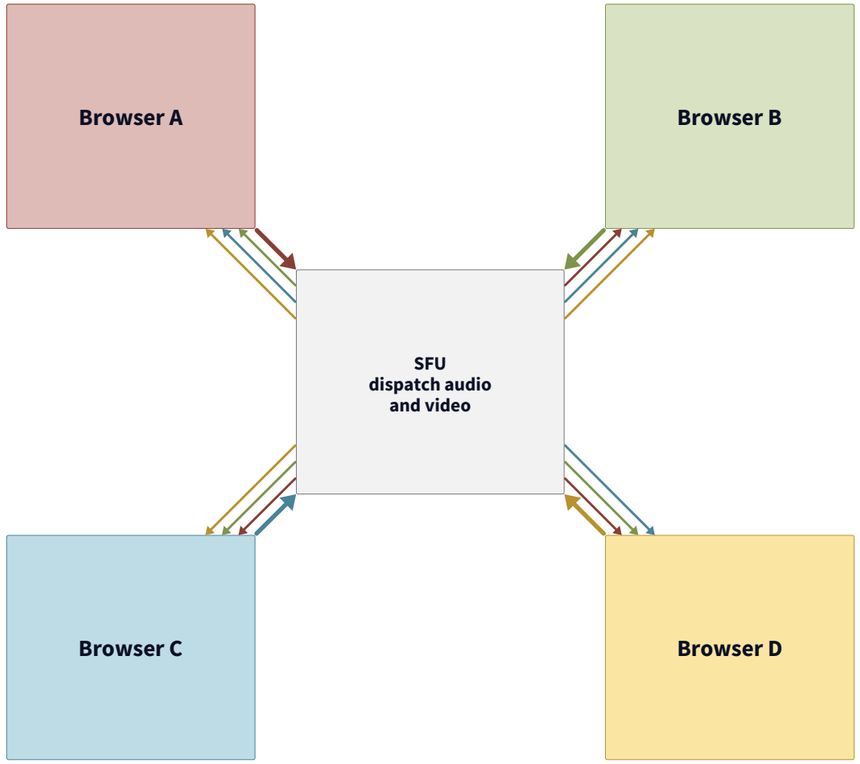
Communication modes



Mesh (P2P)

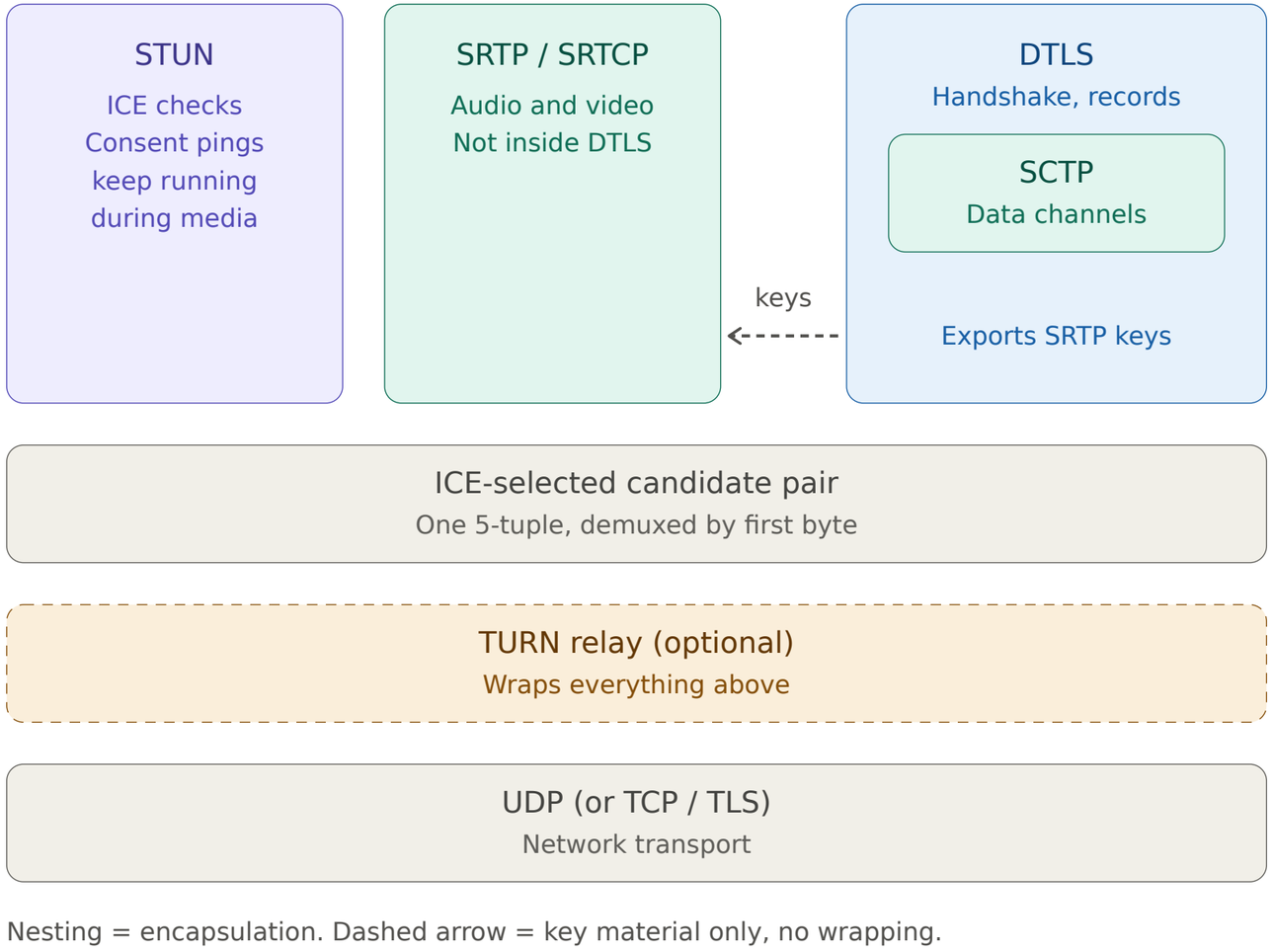


Multipoint Control Unit (MCU)



Selective Forwarding Unit (SFU)

WebRTC network stack



Nesting = encapsulation. Dashed arrow = key material only, no wrapping.

CHAPTER 2

WebRTC call setup in a nutshell

Signaling, ICE, and how a connection is actually chosen

Session Setup Overview

Step	Protocols
Signaling and call control	Websocket (H2), JSON transport, QUIC, SDP
Connection negotiation	ICE using STUN & TURN
Establish media streams	STUN, TURN, DTLS
Secure Transmission	SRTP/SRTCP

> How can we see this?
>
> chrome://webrtc-internals gives you connection sequence step by step

Session Description Protocol (SDP)

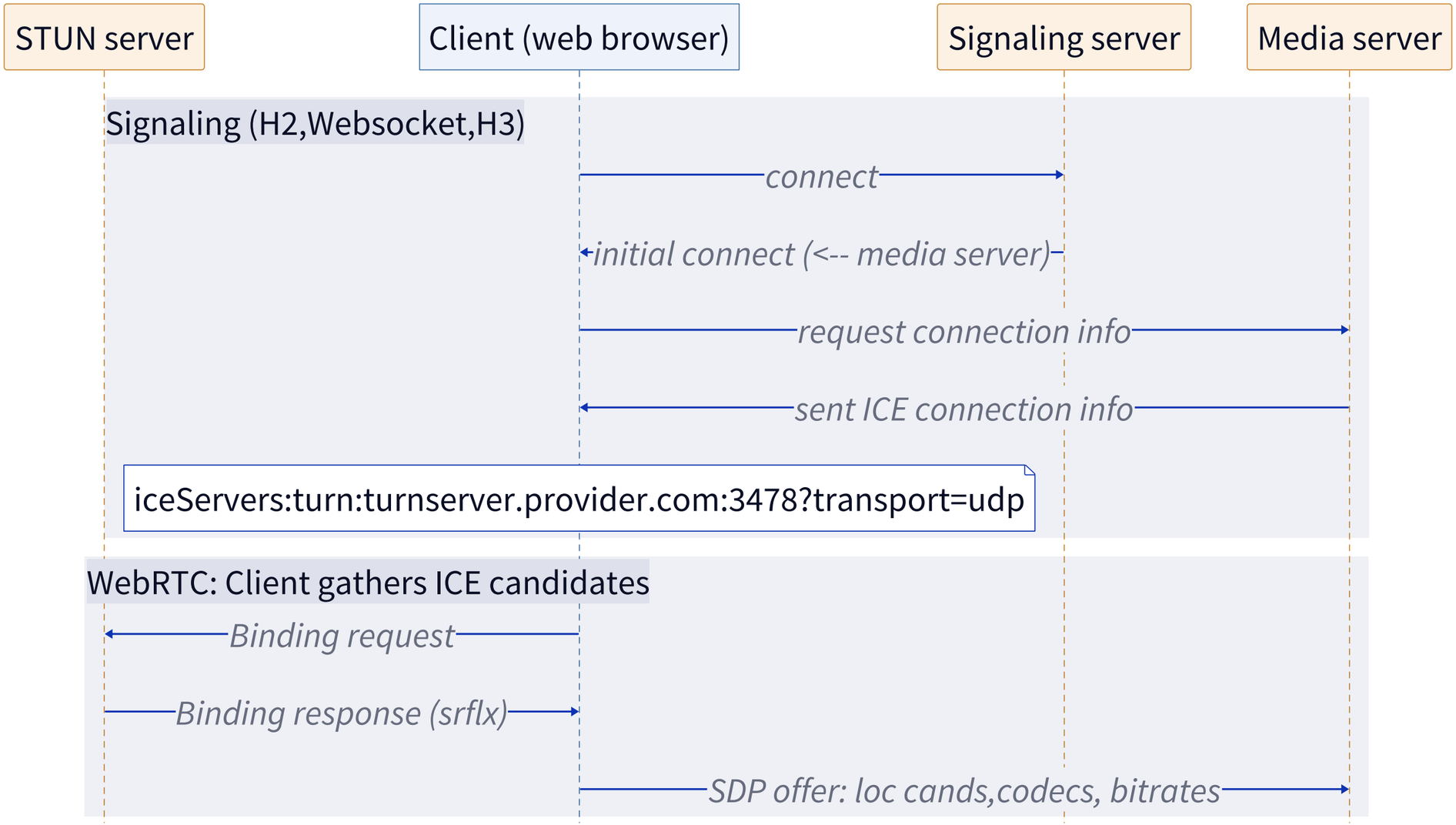
Text format to describe multimedia sessions

Key Purpose: Negotiate codecs, encryption, network addresses, and media directions for peer connection setup.

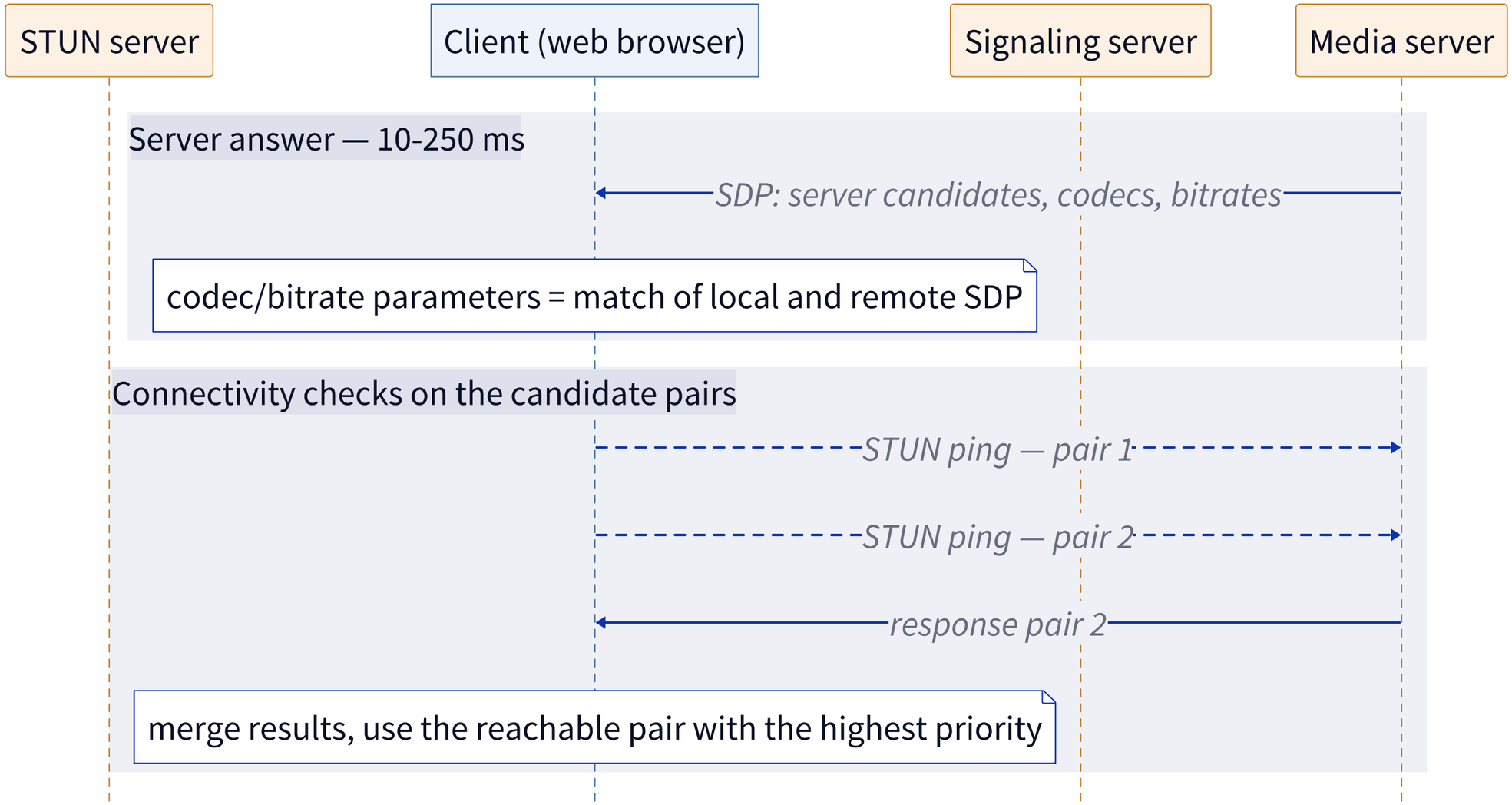
Common SDP Attributes

- **v=** SDP version
- **o=** Session originator
- **m=** Media description (audio/video)
- **c=** Connection information
- **a=** Media attributes
- **rtpmap** Codec mappings
- **ice-ufrag** ICE username fragment
- **ice-pwd** ICE password
- **fingerprint** DTLS certificate hash
- **sendonly/recvonly** Media direction

Signaling - 1


STUN server
Client (web browser)
Signaling server
Media server
Signaling (H2,Websocket,H3)
connect
initial connect (<-- media server)
request connection info
sent ICE connection info
iceServers:turn:turnserver.provider.com:3478?transport=udp
WebRTC: Client gathers ICE candidates
Binding request
Binding response (srflx)
SDP offer: loc cands,codecs, bitrates

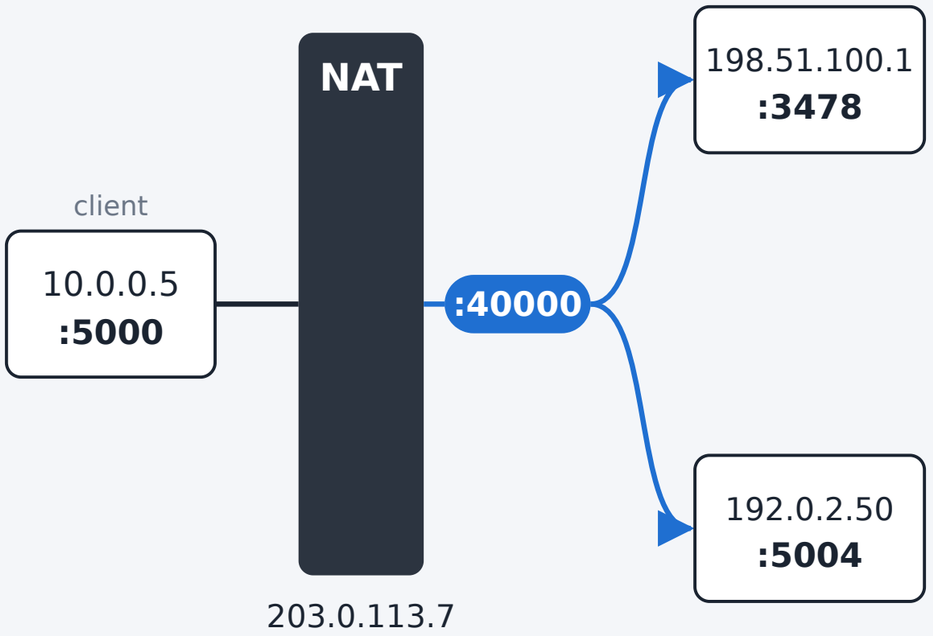
Signaling - 2


STUN server
Client (web browser)
Signaling server
Media server
Server answer — 10-250 ms
SDP: server candidates, codecs, bitrates
codec/bitrate parameters = match of local and remote SDP
Connectivity checks on the candidate pairs
STUN ping — pair 1
STUN ping — pair 2
response pair 2
merge results, use the reachable pair with the highest priority

NAT Types (wrt UDP RFC 4787)

EIM

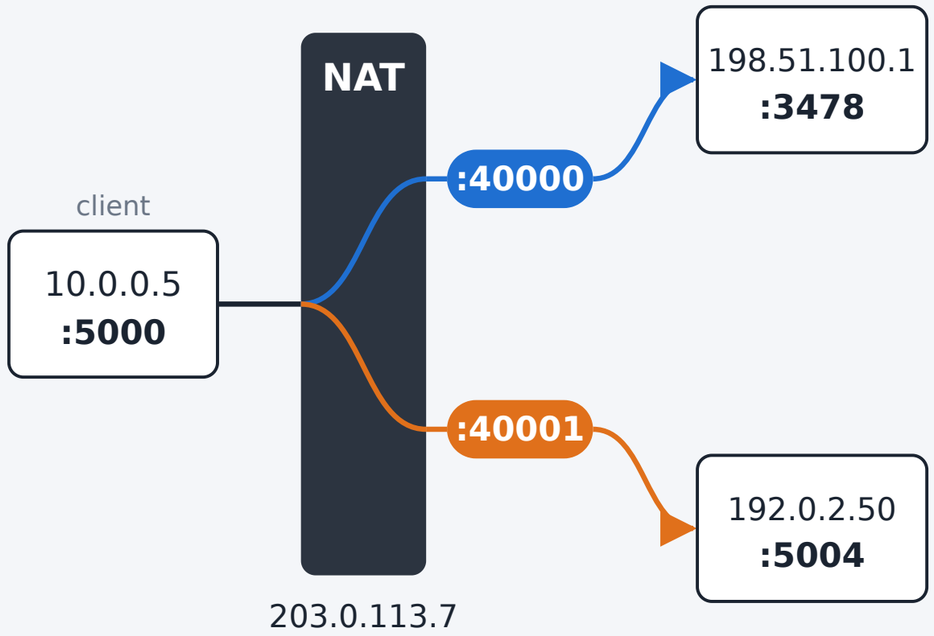
Endpoint-Independent Mapping



one public port per socket

APDM

Address and Port-Dependent Mapping



one public port per destination

ICE Candidates

- Tuple of Id, interface ip, port, type, priority
- Corresponding local and remote candidate form a candidate pair and thus define a connection

see it in the SDP:

a=candidate:1831804035 1 udp 2122194687 192.168.64.1 61885 typ **host** generation 0 network-id 1
a=candidate:2965179438 1 udp 2122129151 192.168.103.91 59672 typ **host** generation 0 network-id 3
a=candidate:1903385444 1 udp *1685921535* **87.142.206.245 25773** typ **srflx** raddr **192.168.103.91** rport 59672 generation 0 network-id 3
a=candidate:2342171461 1 udp 58533119 23.239.237.122 57823 typ **relay** raddr 87.142.206.245 rport 64350 generation 0 network-id 3

OnIceCandidate {“sdpMid”:“1”,“sdpMLineIndex”:0,“candidate”:“candidate:1903385444 1 udp *1685921535* 87.142.206.245 25773 typ srflx raddr 192.168.103.91 rport 59672 generation 0 **ufrag UkSv** network-id 3”,“url”:“stun:turn.goto-rtc.com:3478”}

`remote-ufrag:local-ufrag` is the STUN username, which can be used to tie STUN packets to the corresponding SDP

Client side ICE candidates

depending on detection

Client - local interfaces = host
Client ⟶ STUN server = srflx (server reflexive)
Client ⟷ Media server = prflx (peer reflexive)
Client ⟶ TURN server = relay

Client if:
192.168.101.47 - Local Lan - host
172.17.128.1 - Local HyperV - host

Public (NAT ext) 87.142.206.245 - can be derived via STUN (srflx) or returned from peer (media server - prflx)

TURN server if (relay):
23.239.233.183
3478 UDP
443 TCP (TLS)
5349 TCP (TLS)

Deriving peer connection prios

WebRTC probing to the other party

- Over every interface and address to connect directly
- Via the turn server via various relay options

  → results in candidates and candidate pairs
- This happens from both sides in parallel

Criteria in order of ranking

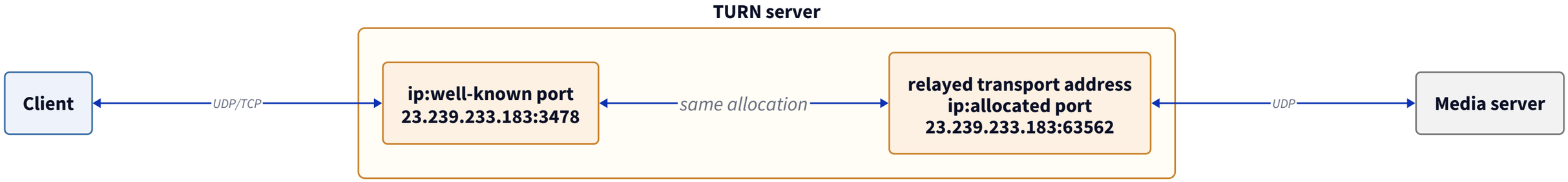
- **type** - how address was derived - *host, prflx,srflx, relay(UDP), relay(TCP),relay(TLS)*
- **network adapter** - Ethernet/wired, WiFi,VPN

> These criteria are layered: *type* is the highest ranking, *network adapter* only counts in when there are two candidates of the same type

More details in Appendix *ICE candidates*

TURN server connectivity

- Well known ports for various protocols
  - 3478 UDP
  - 443 TCP (TLS)
  - 5349 TCP (TLS)
- Listens to inbound TCP & UDP - but second leg is always UDP (high port)
- Each side can use a different TURN, which then send traffic between each other



Signaling - Discussion

- For most client-server use cases this is unnecessarily complex
- You can wire WebRTC and architect your overall system so that it gets easier (see Google Meet)
- Most products use more complex approach (history)

Shortcuts (either configured in the browser or via prudent firewall config)

- Suppress local host candidates
- Do not expose TCP on the server side if they are handled via TURN
- In case of multi-homed or SDWAN, restrict possible path selection

⟶ Fewer candidates make setup faster, easier to debug and expose less information to the outside

> **Shortcuts:**
> **ice-options:trickle** - send SDP right away without any ICE candidates, send more later as soon as they become available - only if still necessary
> **ice-lite**
> - server with public IP only advertises (public) host candidates, no STUN/TURN gathering.
> - server may never initiate STUN pings, just answers them

Media setup

- Connections gets established between negotiated endpoints
- SRTP/DTLS (UDP/TCP) or TLS (TCP) (DTLS/TLS 1.2/1.3)
- Encryption is mandatory
- Regular STUN pings in both direction on the established connection (see ICE Grid)
  - Timeout detection
  - NAT table refresh
  - Indicator for connection health

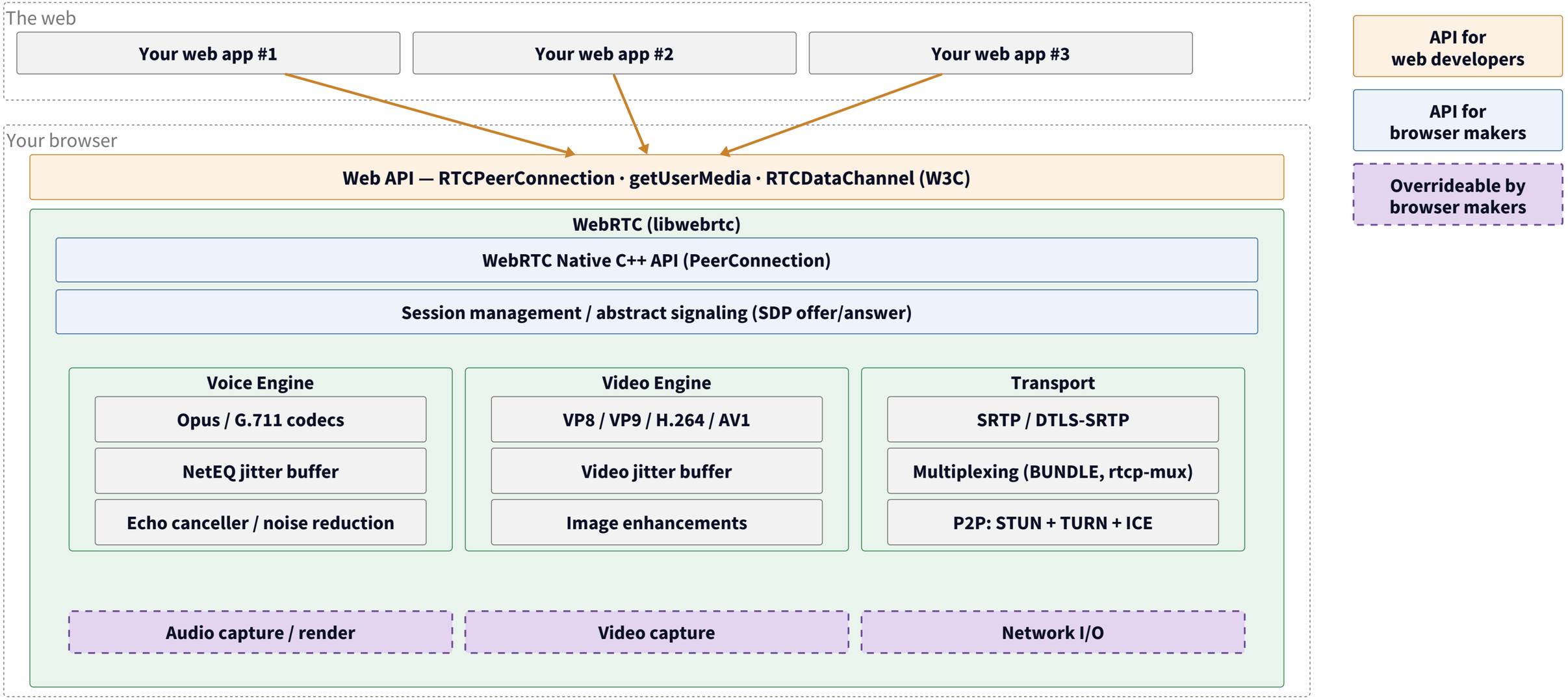
what the packets tell

- Size indicates video or audio
- Media is VBR, but more or less continuous (IO graph)
- Gaps in inbound media indicate network issues
- Gaps in outbound media indicate local device or load issues

Connection setup wrapup

- Initial signaling + WebRTC gives us established media connection
- During call WebRTC regulates bandwidth and handles reconnects
- WebRTC is more than just the connections → comprehensive audio and video signal processing

WebRTC architecture



CHAPTER 3

Analyzing a sample capture

Signaling and media streams unpuzzled



What we have to look at

- User reports/ Notes with points in time (“audio failed after 10 minutes”, “video never connected”)
- Historical evidence (happens always/sometimes)
- Locational evidence (happens when at home connected via VPN - topology)
- Pcapng with or without injected SSL keys (beware when capturing real traffic, use capture filters)
- Application log file (not in browser)
- WebRTC Internals/rtcstats dumps
- ICE grid screenshots
- Maybe Session recording for timing of events or media quality eval
- Interfaces & routing config of machine

Prerequisites

- Wireshark 4.6.x
- Load provided capture file ⇒ goto-full-session-start.pcapng https://bit.ly/4mOt7oR
- Load provided WebRTC Wireshark Profile https://github.com/rohess/wireshark_profiles/tree/main/sfeu26 - this brings:
  - Sets *all* RTP & RTCP & STUN & TURN enabled in `Analyze -> Enabled Protocols` - filter for RTP
  - Filter buttons
  - Color rules
  - `Preferences -> Protocols -> WebSocket -> Dissect websocket text as -> JSON`
- Check `Statistics -> Capture File Properties` for Interface and Decryption secrets
- Enable `View -> Name Resolution -> Resolve Network Addresses`
- Download the Lua plugin for the product from https://github.com/rohess/scripts/tree/main/webrtc-extract-sdp/wireshark
- `Analyze -> Reload Lua Plugins` → Switch to Wireshark

Got a capture - what now

- Open capture - check for `stun` if it's not in, something went wrong
- Check basics (# of packets,IPv4/6, duration, IF, top talkers, decryption secrets, cpu speed, start of call in capture) - clean up if needed (denoise filter from Profile)
- `Statistics -> Conversations`, check `Name resolution`:
  - Where are the large streams (Bytes) TCP or UDP ? (aka is media on UDP or not)
  - If media on TCP - figure out why (are UDP STUN reqs not answered? Proxy CONNECT requests)
  - If media on UDP - check whether your TCP top talkers actually could contain signaling (DNS) if not check SNI in Client Hello
- Filter for `tcp.stream`
  - Check TLS setup - can we decrypt or is it ECH
  - Follow Stream e.g. websocket - ideally we should find here the SDP - this can be wrapped several times - the right Lua plugin will show it to you - you can filter for `sdp`

Media Streams

Same for UDP `Statistics -> Conversations`, check `Name resolution` - top talkers gives you the media servers

- Filter for `udp.stream`
- Audio vs. Video/Screenshare - you could use RTP SSRC from the SDP or just check packet size - audio is typically 50-150 Bytes/packet, video more than one packet per frame
- Number of streams - there are different options of what to do with the down streams (what client receives) - single stream multiplexing it all/ receiving a number of streams from SFU

Alternative Procedure

Alternatively - if you have STUN packets, you know when WebRTC got initialized - setup needs to be before

- filter for `stun||rtp`
- pick frame # of first RTP packet, filter for `frame.number <= #` - this will cutoff the tail, and give you a rather short capture
- `File -> Export Packet Dissections -> As Plain Text`
- Feed packet export to your favorite LLM. Prompt:

> attached is a pcap exported as txt dissection. This captures the start of a meeting. What can you tell me about the session setup, specifically can you find the initial SDP

Live demo of WebRTC Internals

chrome://webrtc-internals
edge://webrtc-internals
firefox - about:webrtc. safari - devtools.

Components

One tab per peer connection+ one for `getUserMedia/getDisplayMedia`

Per peer connection

- Connection String
- ICE connection state
- Active candidate pair
- ICE Grid
- Timeline of connection setup
- Statistics
- Diagrams

How to analyze webrtc-internals and rtcstats dumps

Both can be exported from `chrome://webrtc-internals/`

Difference:

webrtc-internals dump - older format, stats data capped, timestamp correlation not always perfect

rtcstats dump - newer , more efficient format, delta compressed stats, can hold data from long sessions, better timestamp correlation between events and stats

Analyzers

https://rtcstats.github.io/rtcstats/dump-importer/ - free - gives you essentially what you see on the browser page.

https://rtcstats.com/ - very elaborate version, you need to create an account, free and paid version - free version does not have AI analysis

You can self-host if you want.

Bonus



AI Setup for plugin development

- Local Linux VM with tshark installed
- Claude Code via remote-ssh plugin from VS Code towards this Linux VM
- Claude Code for VS Code runs command from VS Code window on the Linux VM
- tshark can be used to read sample captures and also to verify the built plugin
- Plugin can be also be built independently from execution (e.g. in claude.ai) but then you cannot debug it - and sometimes you need to.

Workflow:

- high level analysis of text export in claude.ai → handover md
- handover doc + pcap → Claude Code → creates & tests Lua plugin

CHAPTER 4

Media Performance

Packet loss, delay, TCP, UDP

What do we see without decryption

- The whole ICE/STUN setup (but not the SDP)
- Whether UDP or TCP was used
- Proxy use - ask for DPI
- DNS failures
- Gaps or excessive Jitter in the UDP media stream ← SRTP header (SSRC, sequence number, timestamp) is not encrypted, but for audio packet rate is already telling
- Excessive TCP retransmission

UDP vs TCP

- UDP: unreliable, no retransmission (aka no extra delays)
- TCP: reliable, extra delay when retransmission kicks in, grows with RTT
- Media can deal with quite a bit of loss
  - Forward Error correction - FEC
  - Smart retransmission on codec level
  - Filling the remaining gaps (20 resp 33 ms) with either the same frame again (audio and video) or noise (audio)

> UDP preferred, TCP as fallback

- TCP works best over short RTT and little packet loss - not ideal for transcontinental
- Regulations may require TCP over Proxy - look for a global provider with local onramps (e.g. TURN servers)

Where are the servers? - enable GeoIP in Wireshark


```
Source Address: us05st1.zoom.us (170.114.46.1)
Destination Address: 192.168.101.69 (192.168.101.69)
> [Source GeoIP: US, ASN 209242, Cloudflare London, LLC]
```

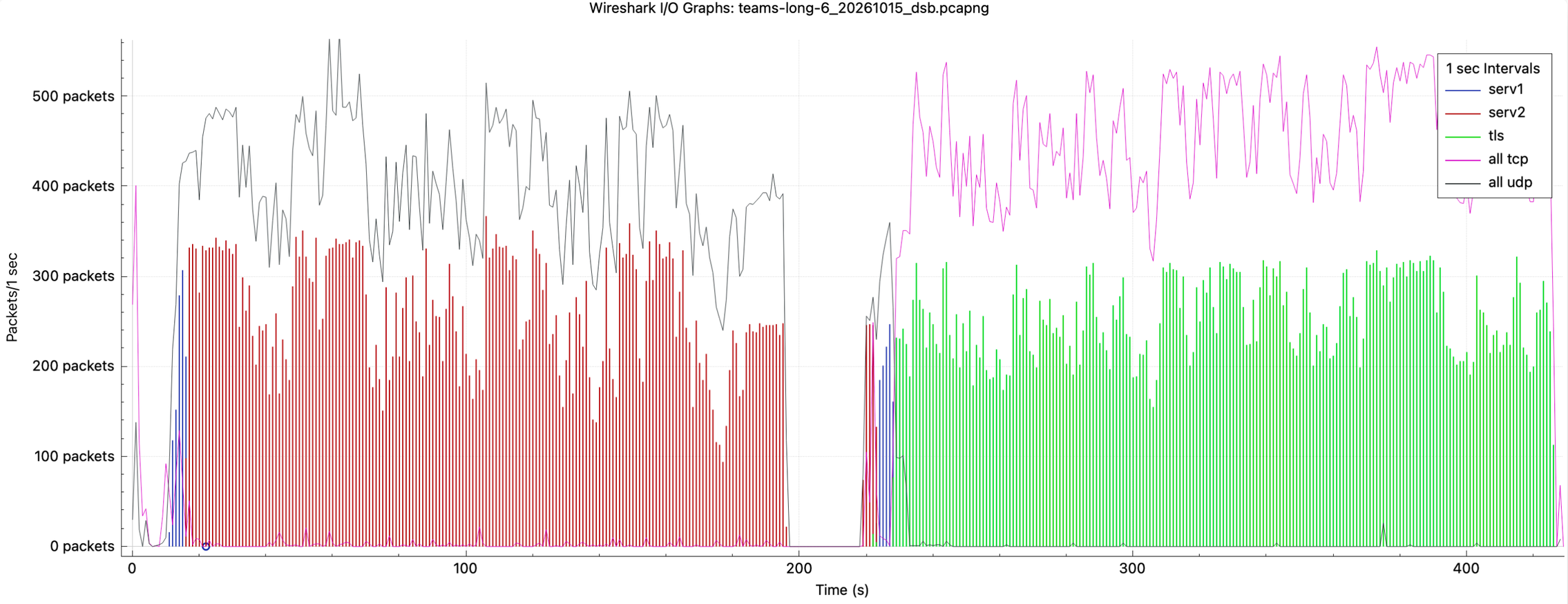
```
Source Address: zoomiad20624799116rwg.iad.zoom.us (206.247.99.116)
Destination Address: 192.168.101.69 (192.168.101.69)
v [Source GeoIP: US, ASN 30103, ZOOM-VIDEO-COMM-AS]
    [Source GeoIP Country: United States]
    [Source or Destination GeoIP Country: United States]
    [Source GeoIP ISO Two Letter Country Code: US]
    [Source or Destination GeoIP ISO Two Letter Country Code: US]
    [Source GeoIP AS Number: 30103]
    [Source or Destination GeoIP AS Number: 30103]
    [Source GeoIP AS Organization: ZOOM-VIDEO-COMM-AS]
    [Source or Destination GeoIP AS Organization: ZOOM-VIDEO-COMM-AS]
    [Source GeoIP Latitude: 38.6583]
```

Look at streams

- Use `Statistics -> Conversations` to build a filter with src.addr and dst.addr for the internal address
- Open `Statistics -> I/O Graphs`
- Duplicate the current filter and edit each so that one has ip.dst - the other ip.src - change the colors
- Set unit to Bits - this will give you bps



Teams call with interrupt



Click to select packet 6136 (22s = 476).

Enabled	Avg over Time	Graph Name	Display Filter	Color	Style	Y Axis	Y Field	SMA Period	Y Axis Factor
☑	☐	serv1	(ip.src == 48.208.184.44)&&rtp		Impulse	Packets		None	1
☑	☐	serv2	(ip.src == 20.202.68.0) &&rtp		Impulse	Packets		None	1
☑	☐	tls	(ip.src == 20.202.0.102) && (tcp.strea...		Impulse	Packets		None	1
☑	☐	all tcp	tcp		Line	Packets		None	1
☑	☐	all udp	udp		Line	Packets		None	1

Typical RTT and Packet loss numbers

RTT from Europe/Germany

Region	RTT [ms]
US East	100-120
US West	160-180
India	200
Satellite (LEO)	+15-40

Packet loss

- Should be single digit - this is what systems can deal with
- Bulk loss is the bigger problem
- Good systems still allow communication at 20-30% average loss over UDP (it’s not nice though)
- Typical sources of high loss
  - Cut undersea cables
  - Overloaded proxies (this is then TCP, even worse)
  - Overloaded local machine (defender & co)
  - Regions with spotty Internet connectivity
  - Cable Internet at prime time

Log files

Log files are interesting because you can retrieve them after the problem happened - a packet capture you have to start upfront

> Generally only desktop apps write log files, browsers can’t.
> See Cheat sheets in appendix for details of the individual products

Browser tools like WebRTC Internals and rtcstats can replace some of this

All products have some user view for Network Quality built in - useful for end user to check

CHAPTER 5

Hands-on Live Capture

Hands-On Lab

Live Capture of WebRTC session

Goal: Capture the WebRTC setup and identify the different components

- Run session in Chrome with TLS key dump
- We will all join a server-based MS Teams session.

Capture Artifacts

- Notes with activity timeline
- pcapng with injected SSL keys afterwards
- Application log file
- WebRTC Internals / rtcstats dumps
- ICE grid screenshots
- Interfaces & routing config of machine

Show of hands

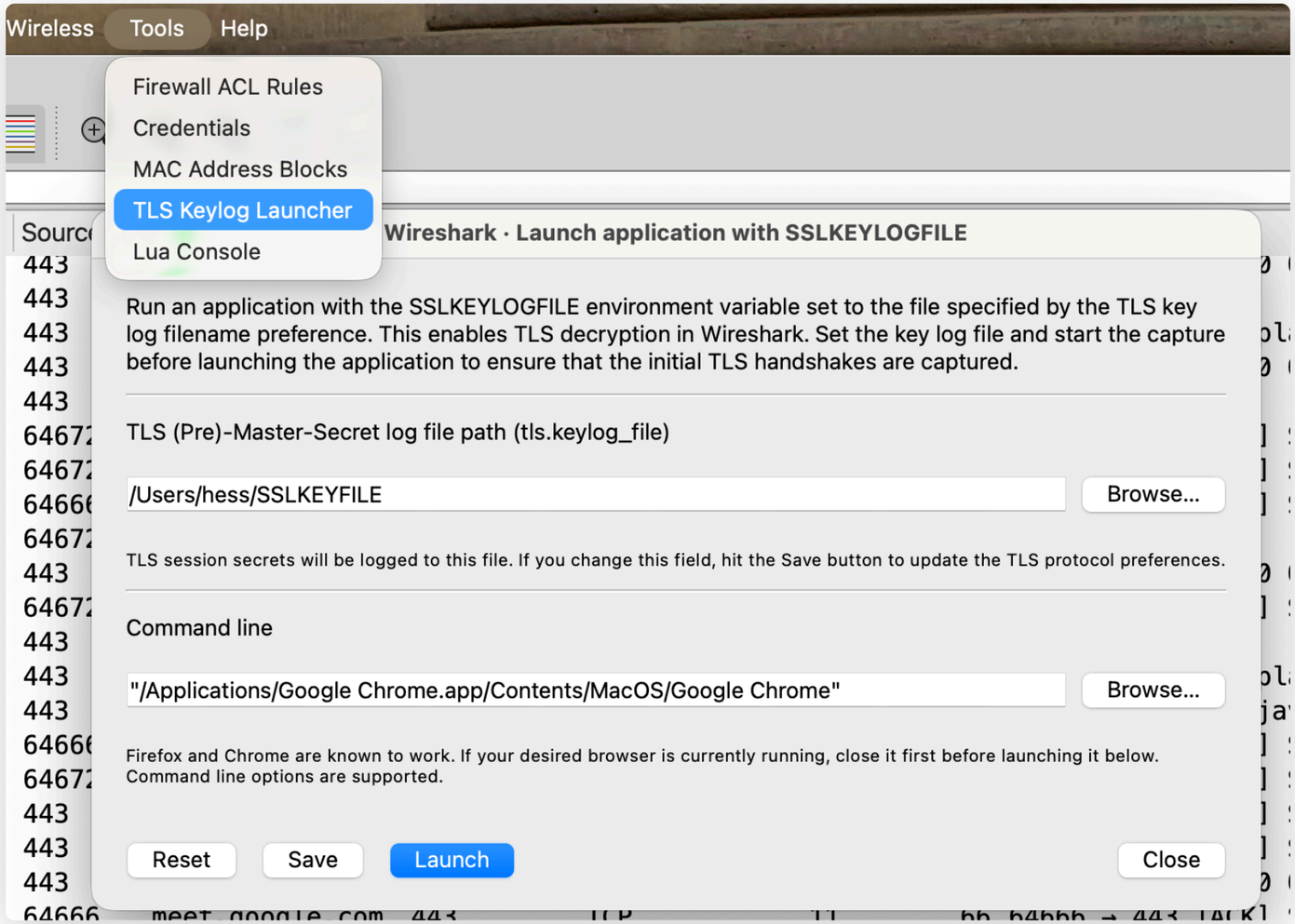
- Who has captured traffic from a Web Browser before?
- Who has done this with decrypting TLS connections?
- Who has worked with the `Statistics -> Conversations` dialog?
- Who is aware of the problems with tunneled DNS/ DNS over HTTPS?

Before we start

- Make sure you have Wireshark >= 4.6 and a recent Chrome/Chromium version (>150)
- In Chrome switch off `Settings -> Privacy and security -> Security -> Use secure DNS` (Wireshark needs to see DNS request)
- Switch off *everything* else (reduce noise)
- Disable OS updates for the next few hours
- Internet connection on
- VPN off – if you can't switch it off, try capture in your VPN client
- If you can, disable all unused Interfaces
- Write down network IF config and routing if it's special
- If not done already - load profile https://github.com/rohess/wireshark_profiles/tree/main/sfeu26
- Make sure you have the right Lua plugin loaded (webrtc-sdp_in_teams.lua)

Run the capture

1. Chrome must **not** be running at all
2. `Tools -> TLS Keylog Launcher`
3. Start Chrome via the launcher
4. Start capture in Wireshark
5. Open chrome://webrtc-internals/
6. Open the Webpage in question
7. Join session - wait for 2 minutes
8. Screenshot of ICE Grid
9. Export rtcstats dump
10. [Cmd|CtrI]+[Option|Alt]+Shift+1 Logs
11. Leave session - Stop capture
12. `Edit -> Inject TLS Secrets`



> Automation: ssldump script

What did we capture



What we have:

Collect all artifacts in a single directory - give it all speaking names

- Screenshots of ICE grid

ICE Candidate pair: 87.142.194.134:13222 <=>142.250.82.220:3478

▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received
CPNO5Ss54x_JnDXz6It	**succeeded**	**ethernet**		**udp**	**0x6e7d1efffe000000**	**5239388 / 381419**
INO5Ss54x	local-candidate	87.142.194.134	13222	prflx	0x6e7d1eff 110 | 32030 | 255	
IJnDXz6It	remote-candidate	142.250.82.220	3478	host	0x7f000004	
CPXBKn1/Og_9/VPLWM7	waiting	ethernet		active	0x5a7d1efffe000000	0 / 0
IXBKn1/Og	local-candidate	192.168.101.34	9	host	0x5a7d1eff 90 | 32030 | 255	
I9/VPLWM7	remote-candidate	142.250.82.253	19305	host	0x7effffe	

- rtcstats/WebRTC Internals dump download analyze using rtcstats.com

Application logfile

- Depends on product, not all have good log files - what there is AI can analyze perfectly

What do we see - TCP

1. Find your own system and rename it to something useful (e.g. CLIENT) `Edit Resolved Name`

> Tip: use upper case for CLIENT and lower case for servers

2. Check `Statistics -> Conversations` - enable `Name resolution`

- TCP – sort by [Packets] – highest count is most interesting – check DNS names

What do we expect:

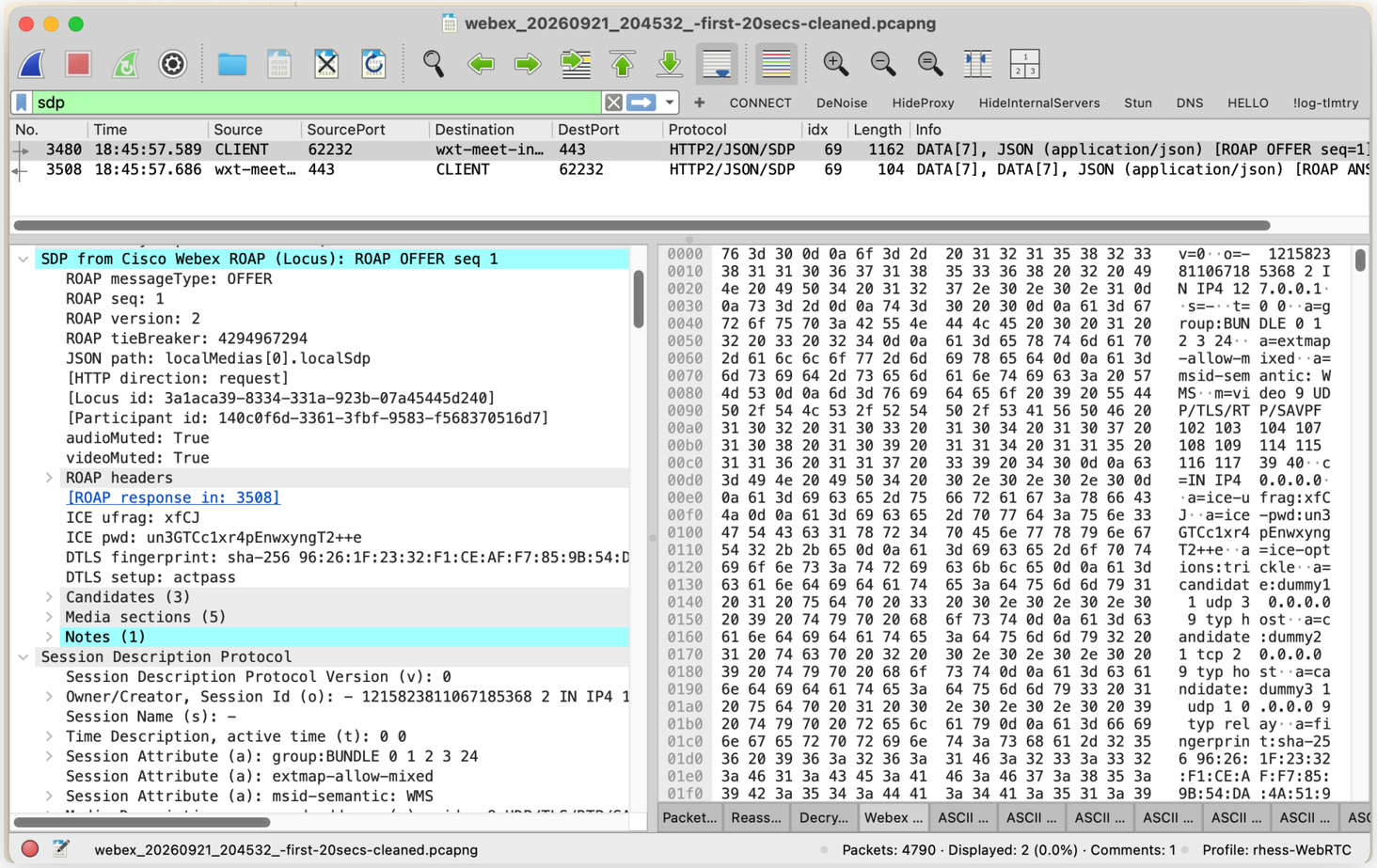
- App download, something from a CDN like Akamai / Cloudfront (this maybe also via QUIC aka UDP)
- Servers with the domain of the meeting provider – product part of domain (`ip.host matches "meet"`)
- Stuff from your browser – for Chrome - Google domains (googleusercontent.com, 1e100.net)
- Connects from your OS (updates etc.) Windows: msedge.net, azure domains

Special cases:

- For proxy check for `http.request.method == "CONNECT"` there are no DNS requests for these servers
- VPN – depending on your type of VPN you will not see the traffic in the capture

TCP analysis

- Check `Statistics -> Conversations` (TCP tab) for top talkers - look at the DNS names → this should give you the signaling connection
- Lua plugin will give you decoded SDP (otherwise you have to scroll endlessly through packet details)



What do we see - UDP

- The iceServers from WebRTC Internals give you a list of servers to use
- Filter for `stun` - is it a single connection or probing towards several endpoints?
- Check in `Statistics -> Conversations` which UDP streams have the most packets - depending on product this is one or several (audio/video)

Salient questions:

- Where are the servers located ? (beware of anycast)
- Do the streams have any gaps? (check packet rate, bandwidth can vary)
- Do we see streams reconnecting (either just a new RTP stream, or full new connection setup)

What did you see different than me

Show my own capture live

Bonus



Setup for clean captures

- Either VM or clean physical machine where lots of things are shut down
  - Auto update (do this manually from time to time)
  - Onedrive, timesync, all multicasting service
  - remove all network stuff except for what you really need (NETBIOS, IPv6)
  - NIC - no checksum offload, segmentation offload
- Connect it via separate USB network IF which is only mapped into VM
- Install only basics (chrome, Wireshark (for dumpcap), text editor)
- Capture via dumpcap script with extra capture

See Appendix for more details.

Capture scripts are here (Windows is the most complete).
https://github.com/rohess/scripts/tree/main/ssldump

CHAPTER 6

Dealing with problems

What could possibly go wrong?



Problem areas

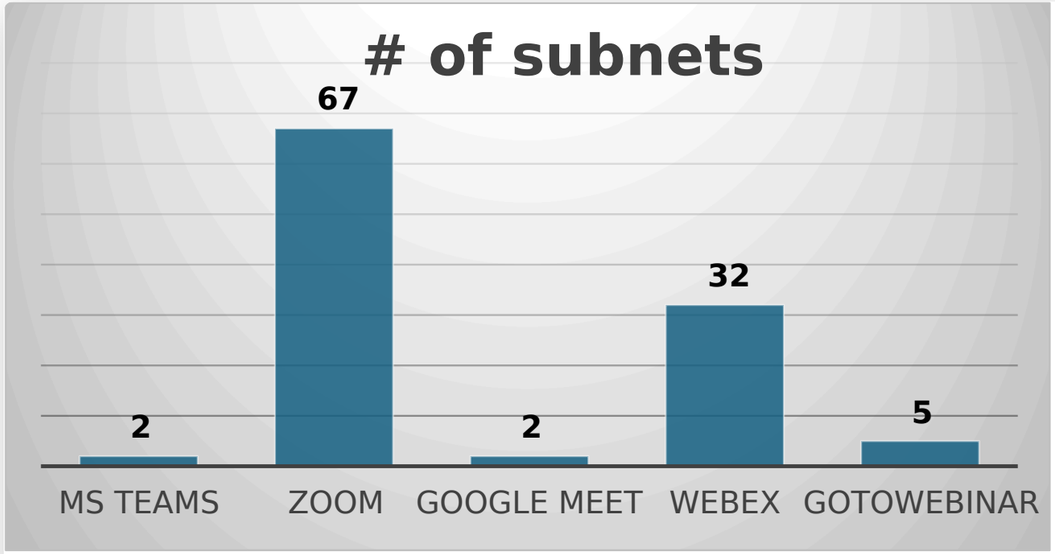
Security vs. Access

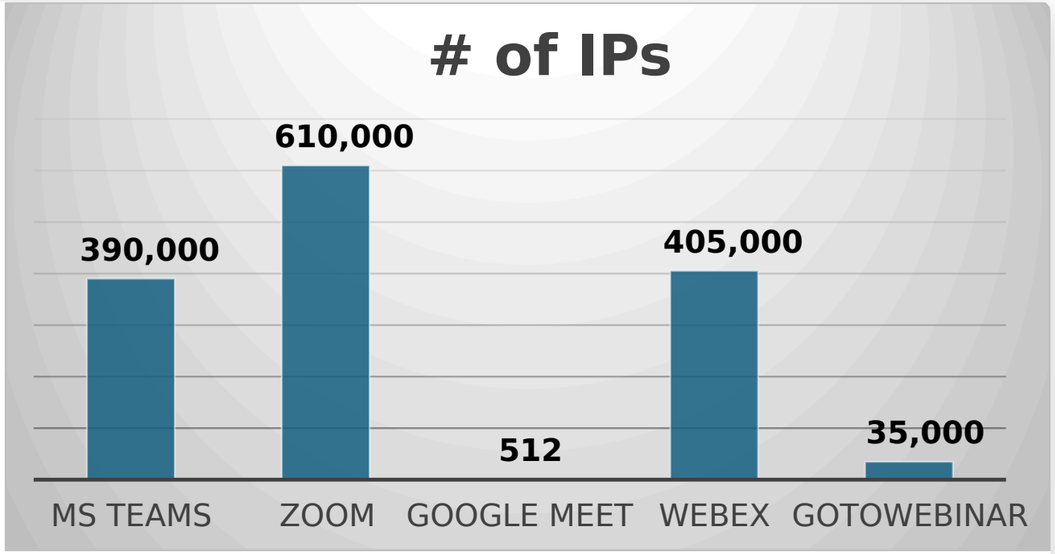
- Dynamic large IP and ports ranges
- Dynamic server DNS names (this is the cloud)
- If companies get acquired, also DNS domains may change
- So in order to configure allow lists for
  - IP:ports (Packet filters)
  - DNS domains (proxies)

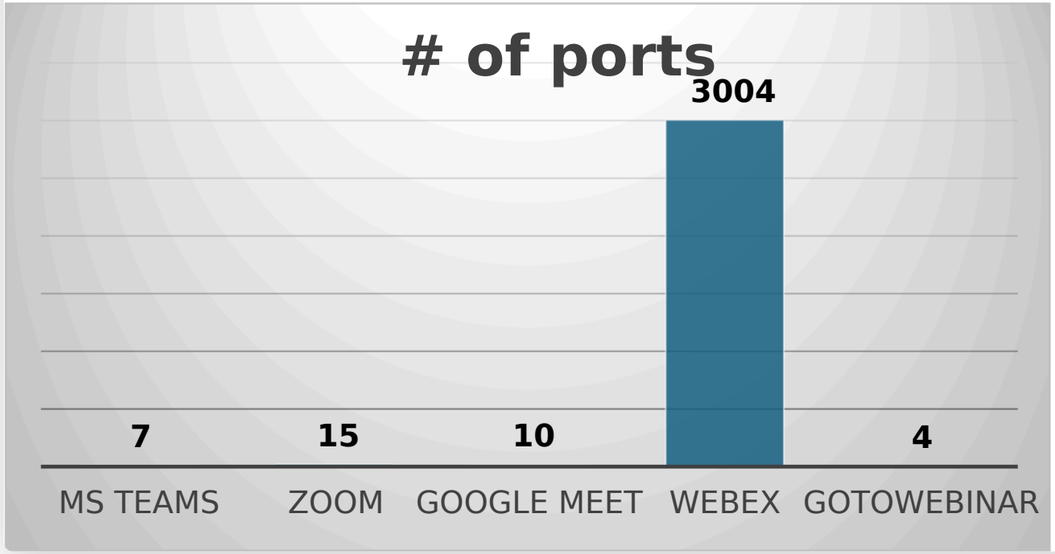
→ We need some automation

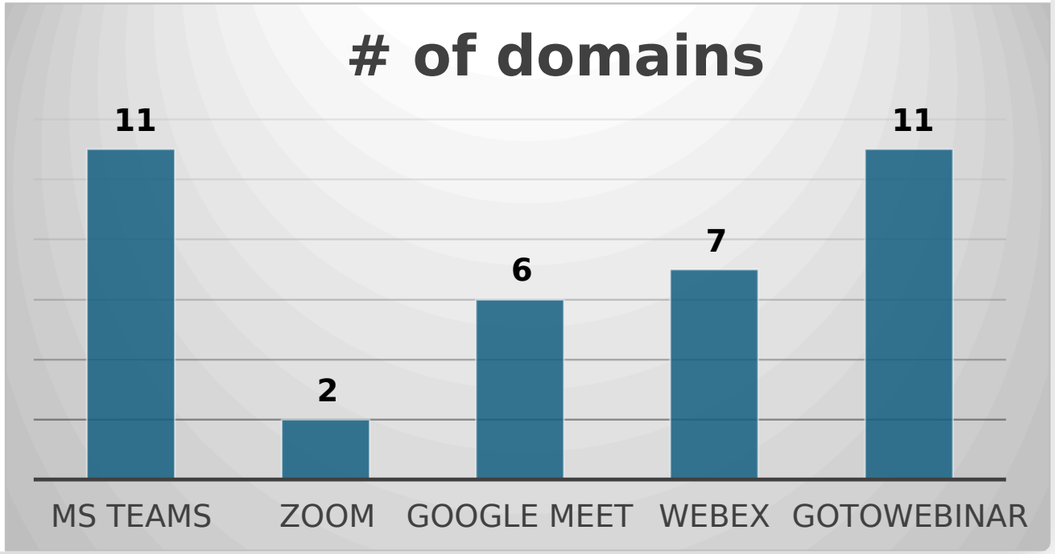
- We also see we ~~cannot~~ should not filter UDP based on DNS (there are evil practices, which I will not recommend)

Network footprint of products









Automate config

Ideally providers have a webpage where your infrastructure can fetch the data:

https://goto-ipblocks.live.gtc.goto.com/

https://endpoints.office.com/endpoints/worldwide?clientrequestid=b10c5ed1-bad1-445f-b386-b919946339a7

see: http://aka.ms/ipurlws

> Most firewalls provide means to automatically fetch & parse these to extract the relevant info.

- Always up to date.
- Security risk if vendor makes mistakes or someone spoofs the page.
- Allowlist whole ASNs ?? Opens a very wide door.

Deep packet inspection

Only on signaling - not on media streams (in case they go over TCP)

- Otherwise easy to overload proxy
- Problem - TURN server misuse to exfiltrate data - not caught by standard DPI - its acting as a UDP relay
- Mitigation - extra rules on endpoints to allow/block such traffic based on originating app.
- Trend to combine network config and endpoint config to get the best results

> **DPI for QUIC**
> Browser will stop sending QUIC when a https proxy is configured
> No simple alternative other than forcing H2 over TCP
> Modern Endpoint Detection & Response products interfere at kernel level and tap into the stream there

VPN, SDWAN, Zscaler, DNS, ICMP

- If possible, break out media traffic locally (performance & load)
- Signaling traffic going through VPN might skew geo selection of servers (similar to tunneled DNS)
- Zscaler & SDWAN flapping between locations gives reconnects & media gaps in meetings
- External DNS on pure VPN machines behind proxies still required
- You need e2e ICMP - at least “needs fragmentation”, missing ICMP may impact TCP and TURN/TLS through VPN overhead.

Checklist for troubleshooting

1. Does not connect at all
2. Connects but bad quality

1.5 → Works only sometimes

Problems are mostly in corporate networks or remote work scenarios. Plain home router setup works 99%.

> If stuff works sometimes or has intermittent drops, its usually about some kind of timeout, different path selection or weird MTU size issues
>
> - Does location or VPN on/off matter
> - Time of the day (load patterns)
> - SDWAN or other variable path selection (Zscaler)
> - Background load on machine

Checklist: No connection

- Signaling connection
  - may fail due to proxy/DPI/certificates `http.request.method == "CONNECT"`
  - plainly blocked (RST after SYN - if you are lucky) `tcp.connection.rst`
- DNS errors (for instance in VPN setup)
- ICE/STUN fails no answer to any of the binding requests - `stun`, check ICE grid
- TURN connection fails look for “Allocate requests” `stun`
- Media connection fails after a few packets `stun||rtp` - heuristic packet filter (Zscaler)
- Connection setup massively delayed (timeouts, also check logs for timeouts)

> Identify failing stage and drill down

Checklist: Bad Media

Machine

- Get either CPU stats or machine specs (Capture file Properties)
- If its running above 80% CPU or has a CoreDuo equivalent CPU, its likely the machine

Network

- 90% it’s TCP over bad connection - check for UDP vs TCP Media traffic (top talkers for UDP/TCP in conversations)

> Identify media path and drill down on it

Checklist: Bad Media # 2

- If TCP
  - Is this intended? Is a proxy involved? Does it DPI the media streams (check TLS certs if you have keylog and they are not encrypted via ECH)
  - Check for retransmissions /dup acks / keep alives - this indicates packet loss and delay `tcp.analysis.retransmission` `tcp.analysis.lost_segment` `tcp.analysis.duplicate_ack` `tcp.analysis.keep_alive`
- If UDP
  - Check stream duration - does it reconnect?
  - Use `Telephony->RTP->RTP Streams` & `RTP Stream Analysis` to drill down into specific stream properties
  - Build IO graphs to see dips in parameters like packets/sec, bits/sec, missing sequence numbers

Security Questions

- How to restrict network footprint optimally
  - uUse access through single port and small IP range (force Turn)
  - Does not work for all products
- How to restrict/monitor TURN server access
  - App level admission on the endpoint
- How to make sure WebRTC does not deviate from the right path
  - Browser config to allow only some types of candidates (Chrome policies `WebRtcIPHandling` and `WebRtcUdpPortRange`)
  - Packet filter allow only specific ranges/ports on the respective paths (VPN gateway could block media traffic, to force local breakout)

Non network:

- File transfer and textchat can exfiltrate data through user action, which is not monitored/audited the way e.g. email is
- block unwanted features in the product

Help the user to help themselves

- All products have some kind of health stats available to the user
- Gives you at least a rough idea
- Needs users who know at least what “bandwidth” is.
- Running speedtest only helps if you get negative results as it uses different protocols and servers
- Educate users (WiFi/Cellular coverage), closing background apps, not using display *link* docks

Bonus



New developments

> There are recent extensions to WebRTC to improve all these things for VoiceAI use case

1. fixed entrypoint for media and pre set media types allow for immediate connection setup without negotiation
2. extra header uniquely identifies connection (similar to MS-TURN) which allows multiplexing thousands of connections over a single (anycast) IP and single port, handover between servers in case of failure or dynamic load balancing
3. Initial STUN binding gets combined with DTLS/TLS setup in QUIC to allow for 1 RTT Media setup

IETF Draft STUN Protocol for Embedding DTLS (SPED)

https://datatracker.ietf.org/doc/draft-hancke-webrtc-sped/

How OpenAI delivers low-latency voice AI at scale

https://openai.com/index/delivering-low-latency-voice-ai-at-scale/

> Will this be used by Webconferencing? → probably not

CHAPTER 7

How products are using WebRTC

Different ways to the same goal

Products we will look at

Most common

MS Teams	Most widely used
Zoom	Runner up, interesting approach
Webex	Quite common

Also interesting

Google Meet	The reference
GoToMeeting	Good to analyze

Appendix has a Cheat sheet for every product including:

- IP ranges & Ports
- Global Distribution
- Connection String
- Sample SDP
- Sample ICE Grid
- Wireshark filter
- Log file location
- Modes of transport

Product peculiarities



Product peculiarities

Each product uses WebRTC in its own, very special way.
Google Meet is the reference

- Modes of transport, one or more media channels, or even data channels
- Connection setup from very slim to full bi-directional p2p-like setup → different reconnect times
- Geo location - DNS, probing lots of servers worldwide for each call, IP Anycast
- Desktop versions may behave differently than the same product in the browser wrt network. (Zoom)
- Support for different browsers differ by product - as browser classes have slight twists in their WebRTC implementation (Chromium/Blink based - Chrome/Edge, Gecko - Firefox, WebKit - Apple) ⟶ Chrome is the reference again
- Worldwide server coverage may differ
- Use of different clouds results in different peerings

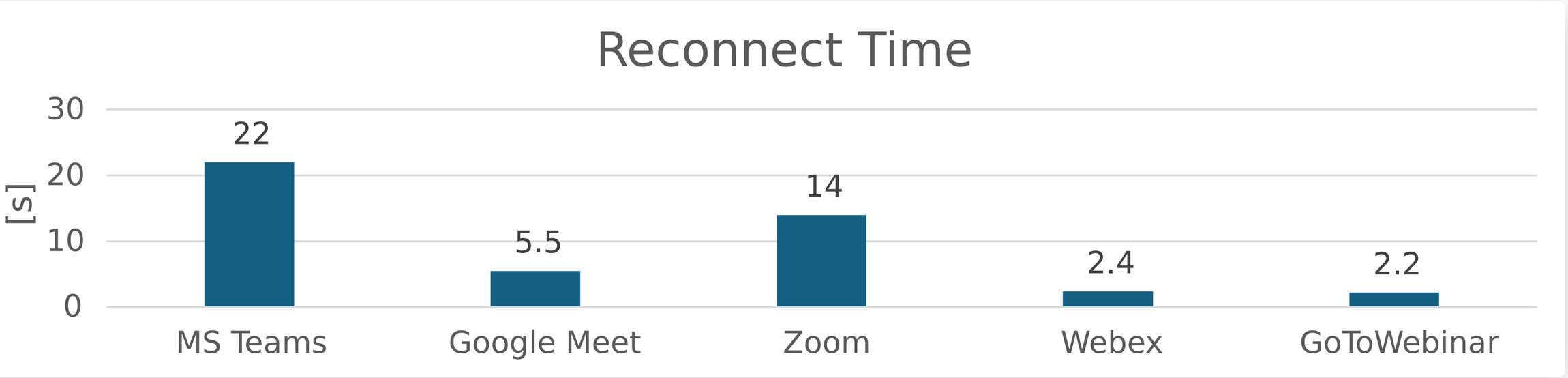
Product Similarities

- All work reasonably well these days and have very similar feature sets, as the vendors copy heavily from each other
- Signaling relies on SDP, regardless of how it’s transported
- Path detection via STUN
- RTP media transport
- Always encrypted
- Max. usable bandwidth around 2.5 Mbps per channel (~ 3Mbps gross)

Desktop versions based on WebRTC usually lag behind browsers, as it takes 1-2 months to pick up latest changes, and Chrome releases every two weeks (major release every 4)

Reconnect behavior (single experiment – just ballpark numbers)

Reconnect when Client IP changes (SDWAN/Zscaler) – aka time without RTP media streams - UDP



Teams	22 s	full setup of peer connection including redirect
Meet	5 s	no ICE , peer connection gets new prflx candidate
Zoom	14 s	full setup of peer connection
Webex	2.4 s	no ICE , peer connection gets new prflx candidate
GoToWebinar	2.2 s	no ICE , peer connection gets new prflx candidate

The individual products



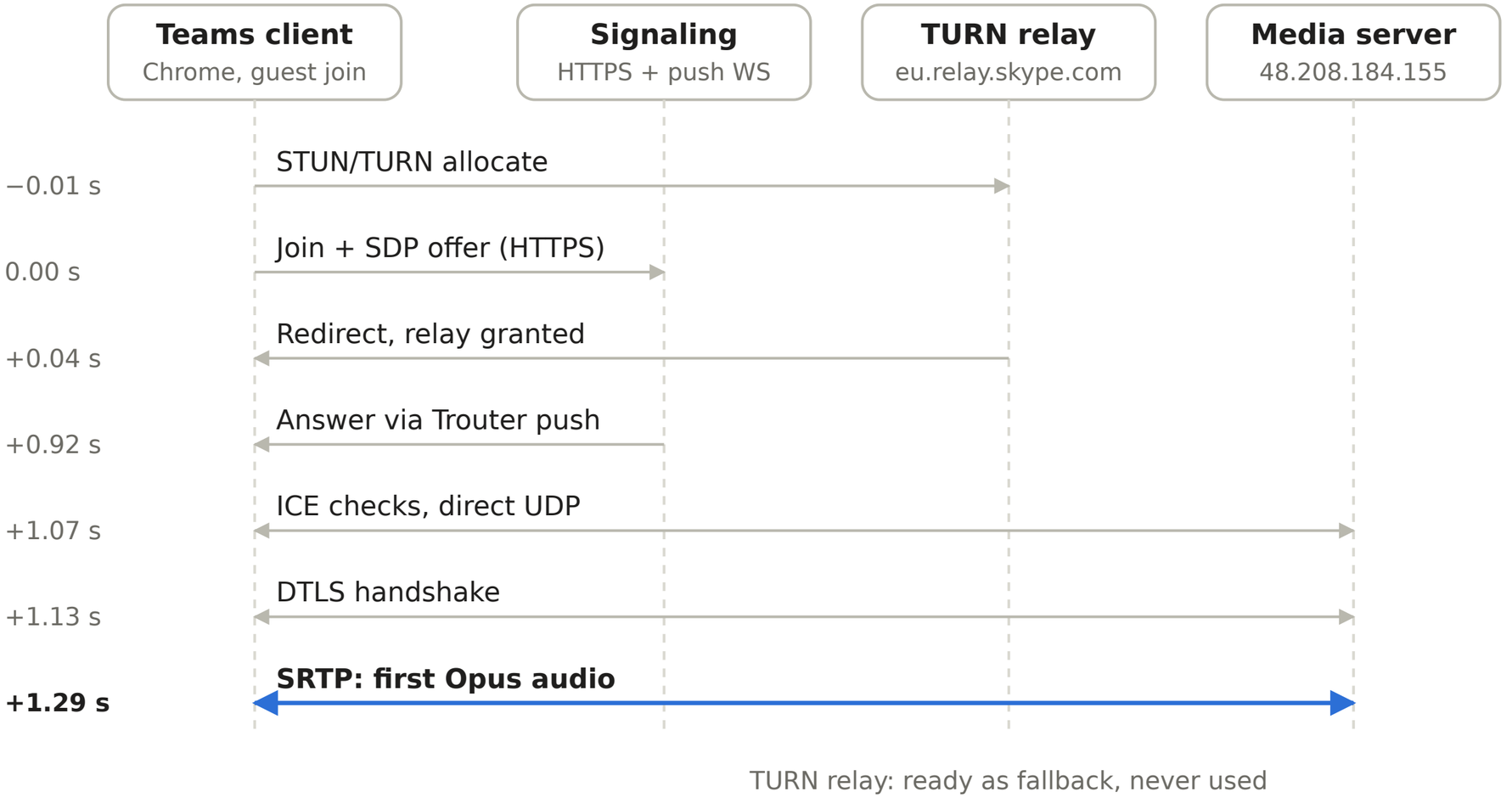
MS Teams

- Runs SDP Offer over different channel (H2/Json) than SDP Answer (websocket)
- Uses their own brand of Turn protocol - MS-TURN
- Multiplexed Turn - allows many clients to send to the same UDP/TCP port and demultiplexes it based on 8 byte prefix
- Supports direct Media flow for 2 party calls - if network allows it. Signaling still centralized.
- H.264 Video, no VP8/VP9
- Native Teams app does not let you decrypt everything (only parts of the app react to SSLKEYLOGFILE)
- First audio flows 1.3 s after the SDP offer, over a direct UDP path

> Seems not all IP ranges used are in the official list, firewall config according to vendor leads to media over TCP.
> You would need to allow UDP 3478 for all Azure ranges

MS Teams connection setup

First audio flows 1.3 s after the SDP offer, over a direct UDP path



Zoom

- Uses separate WebRTC data channels to transport video - audio uses proper audio channel
- Video will thus not show under RTP streams
- Media connections established before join
- SDP wrapped in Zoom’s own framing: binary records containing JSON, inside the WebSocket. The audio SDP is additionally gzip-compressed and base64-encoded
- Desktop app does **not** use WebRTC but Zoom’s proprietary protocol over UDP to port 443 (it’s not QUIC)

Zoom connection setup



Cisco Webex

- Uses *STUN bind request* to 2 ports towards 16 servers to find closest media cluster
- Webex picks the server: the client measures and Webex decides
- SDP over REST, not WebSocket
- Client offer has only placeholder candidates (0.0.0.0 port 9)
- Server uses ICE-lite, sends own addresses: UDP 5004 and 9000, TCP 5004, IPv4, IPv6 and a hostname
- TURN over TLS 443

Cisco Webex connection setup



Bonus

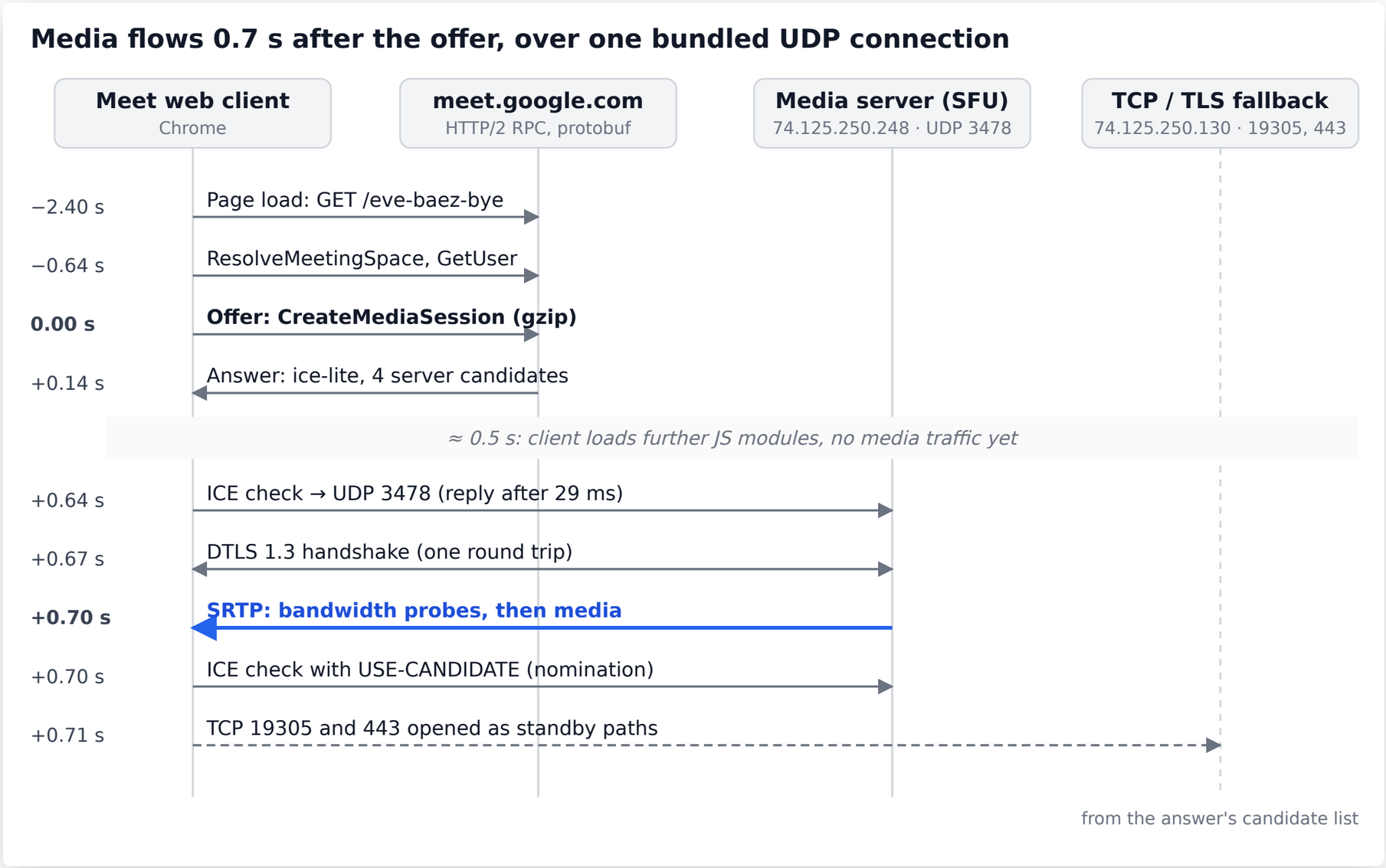


Google Meet

- SDP transferred as gRPC protobuf in a H2 session
- Slightly more cumbersome to extract but doable - see script & Lua plugin
- Server offers UDP 3478, UDP 19305, TCP 19305, TLS 443
- Media flow 700ms after initial offer
- In-session signaling over separate WebRTC Peer connection (data channel).
- No STUN or TURN servers configured in Peer connection it’s all done by “the server”
- Client sends no own candidates, only reads PRFLX candidate – this is ICE Lite.
- Slim setup, but not the best connect/reconnect times.

> Media server IPs are transported inband, and no DNS resolution happens - slightly problematic with TCP proxies, as no SNI is set. But then you can define an IP based rule for proxy as it’s just a single /24 Subnet

Google Meet connection setup



GoToMeeting / Webinar / Training

- All three rely on same frontend code and media servers, backend differs in business logic
- Split infrastructure for audio, video and screensharing → separate peer connections
- GeoDNS for locality
- Four speaker channels downstream from SFU, active speakers get multiplexed into these
- Full ICE setup

GoTo connection setup



Thank You!

WIRESHARK

For listening and staying with us. Feedback please!

